

19CSE401 - Compiler Design
Scientific Calculator Using Lex And Yacc

A PROJECT REPORT

Submitted by

BL.EN.U4CSE22004	A. Neeraja
BL.EN.U4CSE22014	Ch. Mounika begum
BL.EN.U4CSE22030	K. Vigneswara Reddy
BL.EN.U4CSE22050	R. Henry Koushal

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



AMRITA SCHOOL OF COMPUTING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560035, November 2025

ABSTRACT

This project presents the design and implementation of a scientific calculator that applies compiler construction principles using Lex and Yacc. Unlike traditional calculators that perform direct arithmetic computations, this system interprets and evaluates mathematical expressions through lexical and syntax analysis, mimicking the behavior of a compiler. The backend, built using Flex (Lex), Bison (Yacc), and GCC, performs accurate computation of arithmetic, trigonometric, logarithmic, and exponential operations. A Flask-based web interface serves as the frontend, allowing real-time user interaction and dynamic result display. The integration of compiler logic with a web application not only enhances functionality but also demonstrates the practical application of system software concepts in developing interactive computational tools. Overall, the project bridges the gap between theoretical compiler design and modern web-based computing, providing both educational value and real-world utility.

Index Terms —

Lexical Analysis, Syntax Parsing, Compiler Design, Lex (Flex), Yacc (Bison), Flask, Scientific Calculator, Expression Evaluation, System Software, Web Integration.

INTRODUCTION

1.1 Introduction

The project “*Scientific Calculator using Lex and Yacc*” demonstrates the application of compiler design principles in developing a functional scientific calculator. Unlike traditional calculators that perform direct computations, this system interprets and evaluates mathematical expressions using Lexical Analysis and Syntax Parsing, similar to a compiler. The backend, developed with Lex, Yacc, and GCC, performs tokenization, parsing, and evaluation, while a Flask-based web interface enables real-time interaction. This integration bridges the gap between theoretical compiler construction and modern web-based computation, making it both practical and educational.

1.2 Motivation

Conventional calculators lack insight into how expressions are processed internally. This project aims to illustrate compiler behaviour by transforming mathematical expressions into tokens and parse trees before evaluation. It helps students and developers understand compiler operations while delivering a user-friendly, web-based computation platform.

1.3 Problem Definition

Traditional calculators perform direct evaluations without compiler-level analysis. This project addresses that limitation by creating a calculator that utilizes Lex and Yacc to tokenize, parse, and compute mathematical expressions, demonstrating compiler techniques in real-time expression evaluation.

1.4 Objectives

- To develop a compiler-based scientific calculator using Lex and Yacc.
- To apply lexical and syntax analysis for expression evaluation.
- To integrate a Flask web interface for real-time interaction.
- To support arithmetic, trigonometric, logarithmic, and exponential operations.
- To ensure accurate computation and robust error handling.

LITERATURE REVIEW

Compiler-based systems and expression evaluators have long been explored in compiler design and language processing research. Early implementations such as the Simple Calculator Compiler using Lex and Yacc [8] demonstrated how lexical and syntax analysis can be effectively combined to evaluate arithmetic expressions. Subsequent works, such as ToY Language Compiler using Lex and YACC [1], expanded on this idea by supporting user-defined syntax and enhanced grammar rules for more complex language constructs.

Further research, including the Compiler for Mathematical Operations using English-like Sentences [2], illustrated how compiler tools can interpret expressions beyond conventional mathematical syntax. Similarly, Pseudocode to Python – A Compiler Approach [4] and C to Python Translator using Lex and Yacc [9] showcased how the same foundational parsing techniques can be applied to source-to-source translation and automated code generation.

Other studies, such as An Empirical Evaluation of Lex/Yacc and ANTLR Parser Generation Tools [5], analyzed the performance, scalability, and accuracy of different parser generators, concluding that Lex/Yacc remains highly efficient for compact domain-specific compilers. Additionally, works like A Systematic Literature Review of Lexical Analyzer Implementation Techniques [7] provided insights into modern lexical analysis methods for optimizing compiler frontends.

Building upon these developments, this project integrates Lex and Yacc to implement a scientific calculator that evaluates arithmetic, trigonometric, and logarithmic expressions through compiler-based computation. It further extends prior research by incorporating a Flask-based web interface, allowing real-time expression evaluation. This unique combination demonstrates the applicability of compiler principles in modern web-integrated computation environments, bridging theoretical compiler design and practical software engineering.

SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Overview

The system is designed to implement a compiler-based scientific calculator that performs mathematical computations using Lex (Flex) and Yacc (Bison). The backend compiler processes arithmetic and scientific expressions by performing lexical and syntax analysis, while the Flask-based frontend enables real-time user interaction. This chapter outlines the software and hardware requirements, followed by the functional analysis and overall system workflow.

3.2 Software Requirements

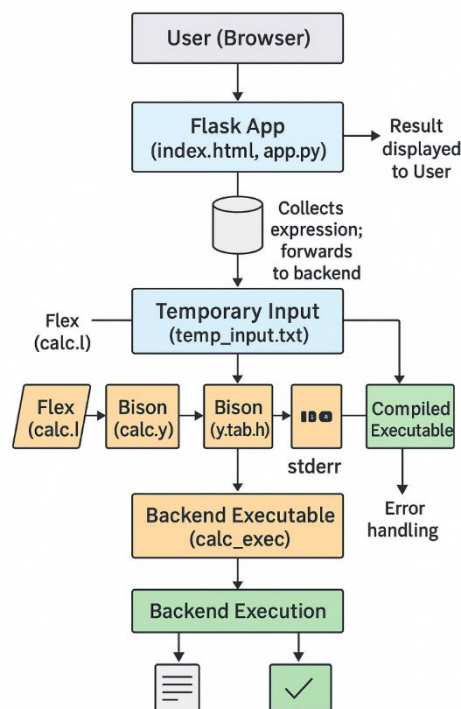
Category	Specification
Operating System	Linux (Kali / Ubuntu)
Programming Languages	C, Python
Compiler Tools	Lex (Flex), Yacc (Bison), GCC
Framework	Flask
Frontend Technologies	HTML, CSS, Bootstrap
Libraries Used	math.h, stdlib.h, string.h
Editor/IDE	Visual Studio Code
Browser	Google Chrome / Mozilla Firefox

3.4 System Analysis

The system is divided into two main components — backend compiler and frontend web interface — that work together to evaluate mathematical expressions in real time.

- Backend (Compiler Layer):
The backend performs lexical and syntax analysis using Lex and Yacc. The input expression is scanned by the lexer (calc.l) to generate tokens such as numbers, operators, and functions. These tokens are then passed to the parser (calc.y), which validates syntax and performs evaluation using semantic actions written in C. The output is computed through GCC compilation and returned as a numerical result.
- Frontend (Web Layer):
The frontend is implemented using Flask, which provides a bridge between the web interface and the compiler backend. Users input expressions on the webpage, which are sent to the backend for processing. The computed result is then dynamically displayed in the browser.

3.5 Work Flow



SYSTEM DESIGN

4.1 System Architecture

The system architecture comprises three main layers:

1. User Interface Layer (Frontend) – Allows users to enter mathematical expressions through a Flask-based web interface designed with HTML, CSS, and Bootstrap.
2. Application Logic Layer (Flask Server) – Acts as a bridge between the frontend and the compiler backend. It sends the user input to the backend, invokes the compilation process, and retrieves the evaluated result.
3. Compiler Engine Layer (Backend) – Built using Lex and Yacc, this layer performs lexical analysis, syntax analysis, and expression evaluation, returning computed results to the Flask server.

4.2 Module Description

- `calc.l` (Lexer): Defines token patterns for numbers, operators, functions, and constants using Flex.
- `calc.y` (Parser): Specifies grammar rules and evaluation logic using Bison, with semantic actions in C.
- `app.py` (Flask App): Handles user input, invokes the compiled backend (`calc_exec`), and displays results on the webpage.
- `index.html` (Frontend): Provides an interactive interface for entering expressions and viewing outputs.

SYSTEM TESTING

5.1 Testing Overview

Testing was carried out to verify the correctness, stability, and performance of the *Scientific Calculator using Lex and Yacc*. The system was tested for module functionality, integration consistency, and computational accuracy.

5.2 Testing and Validation

- **Unit Testing:** Verified token recognition, grammar parsing, and arithmetic evaluation.
- **Integration Testing:** Ensured smooth communication between the Flask frontend and the compiler backend.
- **Functional Testing:** Validated trigonometric, logarithmic, and arithmetic function accuracy.
- **Error Handling:** Checked detection of invalid inputs and division by zero.

RESULTS AND ANALYSIS

The *Scientific Calculator using Lex and Yacc* was successfully implemented and tested. The system accurately evaluates arithmetic, trigonometric, logarithmic, and exponential expressions while maintaining robust communication between the compiler backend and the Flask-based web interface.

The generated outputs matched expected results across all test cases. Both valid and invalid expressions were processed correctly, confirming the effectiveness of lexical and syntax analysis in expression evaluation.

Example outputs include:

- Input: $3 + 4 * 2 \rightarrow$ Output: 11
- Input: $\sin(30) \rightarrow$ Output: 0.5
- Input: $\log_{10}(100) \rightarrow$ Output: 2
- Input: $10 / 0 \rightarrow$ Output: Error: Cannot divide by 0

The system demonstrates how compiler design principles can be effectively applied to practical computation. The use of Lex and Yacc provides reliable parsing and evaluation, while Flask ensures an interactive and user-friendly experience. The project validates that compiler-driven systems can be adapted beyond programming language design to create intelligent, real-time computational tools.

Scientific Calculator



Enter expression (e.g., $\sin(30) + \log$)



C	sin	cos	tan	÷
ln	log	$\sqrt{x}$	$x^{\frac{1}{x}}$	%
n	e	(	)	×
7	8	9	x!	−
4	5	6	x^2	+
1	2	3	EXP	=
	0	.		Ans

✓ $\sin(90) = 0.8939966636$

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

- The project successfully demonstrates the integration of compiler design principles into a functional and interactive scientific calculator.
- By utilizing Lex for lexical analysis and Yacc for syntax parsing, the system accurately interprets and evaluates mathematical expressions.
- The backend compiler efficiently performs computations for arithmetic, trigonometric, logarithmic, and exponential operations.
- The project effectively bridges the gap between theoretical compiler design and practical software application.

7.2 Future Scope

- Extend functionality to support advanced mathematical and statistical operations.
- Integrate graph plotting and visualization for functions and equations.

REFERENCES

- [1] Krishna Vignesh, Mudumba Hanumat, Nallapareddy Vamsidhar Reddy, Sake Mohith Sai, and Meena Belwal. "ToY Language Compiler using Lex and YACC." Sake Mohith and Belwal, Meena, ToY Language Compiler using Lex and YACC (November 15, 2024) (2024).
- [2] Likhith, Arlagadda Naga, Kothuru Gurunadh, Vimal Chinthapalli, and Meena Belwal. "Compiler for mathematical operations using english like sentences." In 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), pp. 1-6. IEEE, 2023.
- [3] Sharma, Ch Rahul AN, Bedadala Mohaneeswar Reddy, Dutta Swetchana, and Niharika Panda. "Compiler Design for recognizing different Programming Languages." In 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), pp. 1-6. IEEE, 2024.
- [4] Mokshit, P., P. Syam Prasad, N. Sumanth Reddy, and Meena Belwal. "Pseudocode to Python-A Compiler Approach." In 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS), pp. 1-5. IEEE, 2024.
- [5] Ortin, Francisco, Jose Quiroga, Oscar Rodriguez-Prieto, and Miguel Garcia. "An empirical evaluation of Lex/Yacc and ANTLR parser generation tools." Plos one 17, no. 3 (2022): e0264326.
- [6] Sridhar, Sashank, and Sowmya Sanagavarapu. "A Compiler-based approach for natural language to code conversion." In 2020 3rd International Conference on Computer and Informatics Engineering (IC2IE), pp. 1-6. IEEE, 2020.
- [7] Pai T, Vaikunta, and P. S. Aithal. "A systematic literature review of lexical analyzer implementation techniques in compiler design." International Journal of Applied Engineering and Management Letters (IJAEML) 4, no. 2 (2020): 285-301.

[8] Upadhyaya, Mohit. "Simple calculator compiler using Lex and YACC." In 2011 3rd International Conference on Electronics Computer Technology, vol. 6, pp. 182-187. IEEE, 2011.

[9] Abhishek, B. Sai, Balam Ruchith Balaji, Chillakuru Hari, and Meena Belwal. "C to Python Translator using LEX & YACC." Available at SSRN 5077711 (2024).

[10] Hahne, Bruce, and Hiroyuki Satō. "Using yacc and lex with c++." ACM SIGPLAN Notices 29, no. 12 (1994): 94-103.